

```

/**
 * server/services/quota-aware-publisher.ts
 *
 * Quota-Aware YouTube Publisher
 *
 * This service makes ZERO AI calls. All metadata, SEO, and thumbnail concepts
 * are already in the DB (written by shorts-prep-pipeline and longform-prep-pipeline).
 * This service only reads 'ready_to_upload' rows and calls the YouTube API.
 *
 * YouTube Data API v3 daily quota: 10,000 units
 * Quota costs:
 *   videos.insert = 1,600 units (upload)
 *   thumbnails.set = 50 units (custom thumbnail)
 *   videos.update = 50 units (metadata update only)
 *   Total per Short: ~1,650 units
 *   Total per longform: ~1,650 units
 *
 * Daily capacity at 10,000 units:
 *   ~6 uploads/day (mix of shorts + longform)
 *   3 Shorts (4,950 units) + 1 longform (1,650 units) = 6,600 units ← safe
 *
 * Quota resets: midnight Pacific Time (07:00 UTC)
 * This publisher checks the reset window and begins uploading immediately
 * after reset, prioritizing the oldest ready_to_upload items first.
 *
 * Schedule:
 *   Checks for ready items every 15 minutes
 *   Uploads begin at 07:05 UTC (5 min after reset, quota always fresh)
 *   Max 3 Shorts + 1 longform per day – enforced via DB counters
 */

```

```

import { createLogger } from '../lib/logger.js';
import { setJitteredInterval } from '../lib/timer-utils.js';
import { storage } from '../storage.js';

```

```

const log = createLogger('quota-aware-publisher');

```

```

// ——— Quota config ———

```

```

const QUOTA_RESET_HOUR_UTC = 7; // YouTube resets at 07:00 UTC
const QUOTA_RESET_BUFFER_MIN = 5; // Wait 5 min after reset before uploading
const MAX_SHORTS_PER_DAY = 3;
const MAX_LONGFORM_PER_DAY = 1;
const QUOTA_COST_UPLOAD = 1600;
const QUOTA_COST_THUMBNAIL = 50;
const QUOTA_DAILY_LIMIT = 10000;
const SAFE_QUOTA_LIMIT = 8000; // Stop at 8k – leave 2k buffer for retries

```

// — Quota window helpers —

```
function getQuotaWindowStart(): Date {
  const now = new Date();
  const reset = new Date(now);
  reset.setUTCHours(QUOTA_RESET_HOUR_UTC, QUOTA_RESET_BUFFER_MIN, 0, 0);
  if (reset > now) {
    // Reset is later today
    return reset;
  }
  // Reset already happened today – window started at reset time
  return reset;
}
```

```
function isInPublishWindow(): boolean {
  const now = new Date();
  const hourUTC = now.getUTCHours();
  const minUTC = now.getUTCMinutes();
  const totalMin = hourUTC * 60 + minUTC;
  const resetMin = QUOTA_RESET_HOUR_UTC * 60 + QUOTA_RESET_BUFFER_MIN;
  // Publish from 07:05 UTC to 06:55 UTC next day (i.e. almost always ok)
  return totalMin >= resetMin;
}
```

// — Main publish function —

```
async function publishReadyShort(
  payload: any,
  youtubeClient: any,
  userId: string
): Promise<void> {
  log.info(`[QuotaPublisher] Uploading Short – clip ${payload.clipId}: "${payload.title}"`);

  // 1. Upload the video file
  const uploadResponse = await youtubeClient.videos.insert({
    part: ['snippet', 'status'],
    requestBody: {
      snippet: {
        title: payload.title,
        description: payload.description,
        tags: payload.tags,
        categoryId: payload.categoryId,
        defaultLanguage: payload.defaultLanguage,
      },
      status: {
        privacyStatus: 'public',
        selfDeclaredMadeForKids: false,
      },
    },
    media: {
```

```

        body: await storage.getClipReadStream(payload.clipId),
    },
});

const youtubeVideoId = uploadResponse.data.id;
log.info(`[QuotaPublisher] ✅ Short uploaded – YouTube ID: ${youtubeVideoId}`);

// 2. Set custom thumbnail if rendered
if (payload.thumbnailFilePath) {
    try {
        await youtubeClient.thumbnails.set({
            videoId: youtubeVideoId,
            media: {
                body: await storage.getThumbnailReadStream(payload.thumbnailFilePath),
            },
        });
        log.info(`[QuotaPublisher] ✅ Thumbnail set for ${youtubeVideoId}`);
    } catch (err) {
        log.warn(`[QuotaPublisher] Thumbnail upload failed for ${youtubeVideoId} – continuing`,
err);
    }
}

// 3. Mark as published in DB
await storage.markShortsClipPublished(payload.clipId, {
    youtubeVideoId,
    publishedAt:    new Date(),
    title:          payload.title,
    description:    payload.description,
    tags:           payload.tags,
    thumbnailConcept: payload.thumbnailConcept,
});

await storage.incrementDailyPublishCount(userId, 'short');
log.info(`[QuotaPublisher] Clip ${payload.clipId} marked published`);
}

async function publishReadyLongform(
    payload: any,
    youtubeClient: any,
    userId: string
): Promise<void> {
    log.info(`[QuotaPublisher] Uploading long-form video ${payload.videoId}: "${payload.title}"`);

    const uploadResponse = await youtubeClient.videos.insert({
        part: ['snippet', 'status'],
        requestBody: {
            snippet: {
                title:          payload.title,
                description:    payload.description,

```

```

tags:      payload.tags,
categoryId: payload.categoryId,
defaultLanguage: payload.defaultLanguage,
},
status: {
  privacyStatus:      'public',
  selfDeclaredMadeForKids: false,
},
},
media: {
  body: await storage.getVideoReadStream(payload.videoId),
},
});

const youtubeVideoId = uploadResponse.data.id;
log.info(`[QuotaPublisher] ✅ Long-form uploaded – YouTube ID: ${youtubeVideoId}`);

// Thumbnail
if (payload.thumbnailFilePath) {
  try {
    await youtubeClient.thumbnails.set({
      videoId: youtubeVideoId,
      media: { body: await storage.getThumbnailReadStream(payload.thumbnailFilePath) },
    });
  } catch (err) {
    log.warn(`[QuotaPublisher] Thumbnail upload failed for ${youtubeVideoId}`, err);
  }
}

// Mark published
await storage.markVideoPublished(payload.videoId, {
  youtubeVideoId,
  publishedAt: new Date(),
  title:      payload.title,
  description: payload.description,
  tags:      payload.tags,
  chapters:   payload.chapters,
  thumbnailConcept: payload.thumbnailConcept,
});

await storage.incrementDailyPublishCount(userId, 'longform');
log.info(`[QuotaPublisher] Video ${payload.videoId} marked published`);
}

// — Publish cycle —————

export async function runPublishCycle(userId: string, youtubeClient: any): Promise<void> {
  if (!isInPublishWindow()) {
    log.info(`[QuotaPublisher] Outside publish window – skipping`);
    return;
  }

```

```

}

const windowStart = getQuotaWindowStart();
const todayCounts = await storage.getDailyPublishCounts(userId, windowStart);
const quotaUsed = await storage.getQuotaUsedToday(userId, windowStart);

log.info(
  `[QuotaPublisher] Daily counts – Shorts: ${todayCounts.short}/${MAX_SHORTS_PER_DAY}, ` +
  `Long-form: ${todayCounts.longform}/${MAX_LONGFORM_PER_DAY}, ` +
  `Quota: ~${quotaUsed}/${SAFE_QUOTA_LIMIT} units`
);

if (quotaUsed >= SAFE_QUOTA_LIMIT) {
  log.warn('[QuotaPublisher] Approaching quota limit – pausing until reset');
  return;
}

// — Publish Shorts (up to daily limit) —————

const shortsRemaining = MAX_SHORTS_PER_DAY - todayCounts.short;

if (shortsRemaining > 0) {
  const readyShorts = await storage.getReadyShortsPayloads(userId, shortsRemaining);
  log.info(`[QuotaPublisher] ${readyShorts.length} Shorts ready to upload (${shortsRemaining}
slots remaining)`);

  for (const payload of readyShorts) {
    const estimatedCost = QUOTA_COST_UPLOAD + (payload.thumbnailFilePath ?
QUOTA_COST_THUMBNAIL : 0);
    if (quotaUsed + estimatedCost > SAFE_QUOTA_LIMIT) {
      log.warn('[QuotaPublisher] Quota limit would be exceeded – stopping Short uploads');
      break;
    }

    try {
      await publishReadyShort(payload, youtubeClient, userId);
      await storage.recordQuotaUsage(userId, estimatedCost);
      // Spacing between uploads – YouTube prefers a gap
      await new Promise(r => setTimeout(r, 10000));
    } catch (err: any) {
      log.error(`[QuotaPublisher] Short upload failed for clip ${payload.clipId}:`, err);
      if (err?.code === 403) {
        log.error('[QuotaPublisher] 403 Quota exceeded – halting all uploads today');
        return;
      }
      await storage.markShortsClipUploadFailed(payload.clipId, err.message);
    }
  }
}
}

```

```
// — Publish Long-form (up to daily limit) —————

const longformRemaining = MAX_LONGFORM_PER_DAY - todayCounts.longform;

if (longformRemaining > 0) {
  const readyVideos = await storage.getReadyLongformPayloads(userId, longformRemaining);
  log.info(`[QuotaPublisher] ${readyVideos.length} long-form videos ready
(${longformRemaining} slots remaining)`);

  for (const payload of readyVideos) {
    const estimatedCost = QUOTA_COST_UPLOAD + (payload.thumbnailFilePath ?
QUOTA_COST_THUMBNAIL : 0);
    if (quotaUsed + estimatedCost > SAFE_QUOTA_LIMIT) {
      log.warn(`[QuotaPublisher] Quota limit would be exceeded – stopping long-form uploads`);
      break;
    }

    try {
      await publishReadyLongform(payload, youtubeClient, userId);
      await storage.recordQuotaUsage(userId, estimatedCost);
      await new Promise(r => setTimeout(r, 15000));
    } catch (err: any) {
      log.error(`[QuotaPublisher] Long-form upload failed for video ${payload.videoId}:`,
err);
      if (err?.code === 403) {
        log.error(`[QuotaPublisher] 403 Quota exceeded – halting all uploads today`);
        return;
      }
      await storage.markVideoUploadFailed(payload.videoId, err.message);
    }
  }
}

log.info(`[QuotaPublisher] Publish cycle complete`);
}

// — Service lifecycle —————

let stopPublisher: (() => void) | null = null;

export function startQuotaAwarePublisher(userId: string, youtubeClient: any): void {
  if (stopPublisher) {
    log.warn(`[QuotaPublisher] Already running – skipping double-start`);
    return;
  }

  log.info(`[QuotaPublisher] Starting – check interval: ~15 min`);

  // Run once immediately on startup to catch items that were ready before boot
  runPublishCycle(userId, youtubeClient).catch(err =>

```

```
    log.error('[QuotaPublisher] Startup cycle error:', err)
  );

  stopPublisher = setJitteredInterval(
    () => runPublishCycle(userId, youtubeClient).catch(err =>
      log.error('[QuotaPublisher] Cycle error:', err)
    ),
    15 * 60 * 1000, // 15 min base, ±20% jitter
  );
}

export function stopQuotaAwarePublisher(): void {
  if (stopPublisher) {
    stopPublisher();
    stopPublisher = null;
    log.info('[QuotaPublisher] Stopped');
  }
}
```